VeriBlue AI – Technical Specification & Product Requirements Document

Multi-Stage AI-Powered File Integrity Verifier

1. Project Overview & Vision

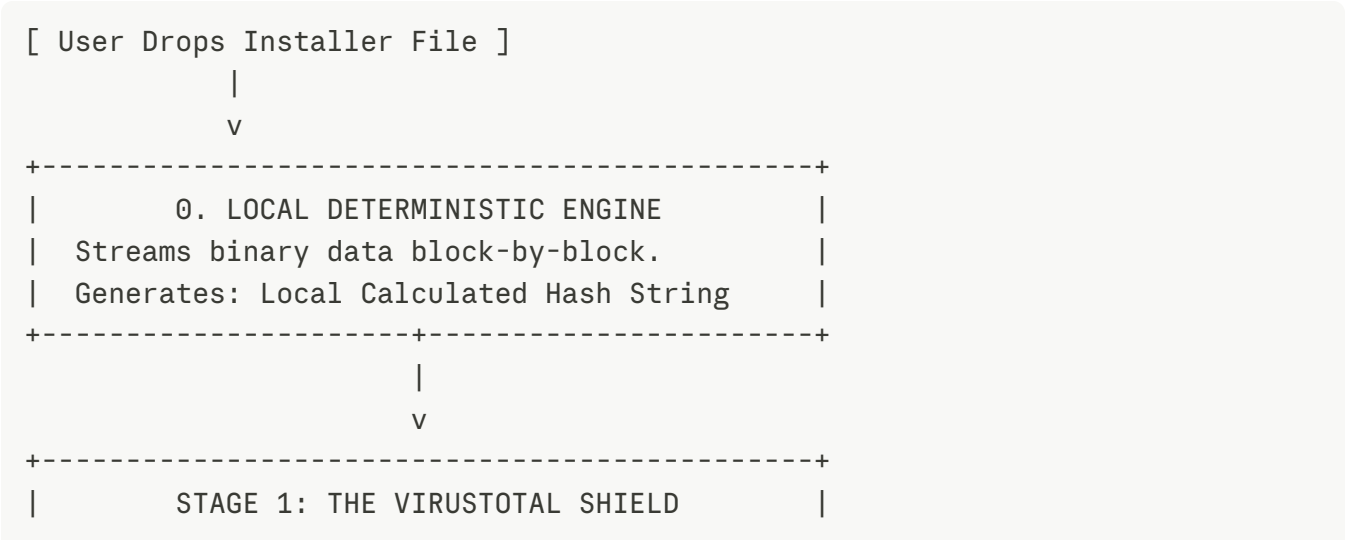
VeriBlue AI is an intelligent desktop security utility that automates the verification of downloaded software installers, application binaries, and deployment archives.

Typical antivirus programs focus on flagging known malware. VeriBlue AI prioritizes **data integrity**. It uses a strict defense-in-depth, multi-stage pipeline:

- **Stage 1 – The Malware Shield:** uses the VirusTotal API as a global threat registry to screen the file's fingerprint against known malicious entries.
- **Stage 2 – The Integrity Shield:** if Stage 1 returns an unknown status, an automated web-search RAG engine crawls the web, and a precise LLM context layer extracts the official author's published verification hash from the vendor's domain. This proves mathematically whether a third-party download has been modified, tampered with, or corrupted.

2. Technical Architecture & Pipeline

The software uses a **deterministic hybrid architecture**. Mathematical calculations are kept separate from language-model processing, so AI hallucination can never corrupt a cryptographic string.



```
|  Sends ONLY the Local Calculated Hash string |
|  as a lookup key to check for known threats. |
+----------------------+-----------------------+
                       |
          (If API returns 404 / Unknown)
                       |
                       v
+-----------------------------------------------+
|      STAGE 2: THE WEB-SEARCH RAG ENGINE       |
|  1. Extracts name & version metadata tokens.  |
|  2. Scrapes text from official vendor domain. |
|  3. LLM extracts the true Fetched Hash.       |
+----------------------+-----------------------+
                       |
                       v
+-----------------------------------------------+
|       3. CRYPTOGRAPHIC MATCH ENGINE           |
|  Simple, absolute boolean logic:              |
|  Is Local Calculated Hash == Fetched Hash?    |
+-----------------------------------------------+
```

Step 0: Local Deterministic Layer (Python Native)

- **Metadata harvesting:** the local engine parses the binary's structural headers (for example, Portable Executable headers on Windows or `Info.plist` XML tags on macOS) to isolate the application name, major/minor release version, and target architecture.
- **Stream hashing engine:** the file is streamed block by block in 4096-byte buffer slices to build a 64-character SHA-256 **Local Calculated Hash**. This prevents memory exhaustion on large installers.

Step 1: Stage 1 Malware Shield (VirusTotal Integration)

- **Fingerprint lookup call:** only the 64-character Local Calculated Hash is sent, as a lightweight text query, to the VirusTotal v3 Files endpoint.
- **Conditional routing logic:**
  - **If Malicious > 0:** execution halts immediately. The raw JSON threat diagnostics go to the AI Explainer, which formats an active warning canvas.
  - **If status code is 404 (Not Found):** the file signature is new, a fresh developer point release, or a private peer file. Execution cascades into Stage 2.

Step 2: Stage 2 Integrity Shield (Web-Search RAG Loop)

- **Domain-restricted query formulation:** the application builds a search query with strict parameters, for example `site:videolan.org VLC Media Player 3.0.20 SHA256`.
- **Scraping and context caching:** a RAG web retriever (Tavily Search API) crawls the authenticated vendor URL, strips layout clutter, and stores the text fragments in a temporary, ephemeral in-memory cache (ChromaDB or local memory arrays).
- **AI extraction parser:** a low-cost, precise LLM (`gpt-4o-mini` or `gemini-2.5-flash`) reads the trusted context block and isolates the official **Fetched Hash** string.

Step 3: Cryptographic Match Validation

The backend runs a traditional comparison: `Local Calculated Hash == Fetched Hash`.

- **True:** shows a green confirmation dashboard ("Genuine, clean deployment verified").
- **False:** shows a high-visibility red warning canvas ("Integrity Broken: File has been altered by a third party").

3. Workflow & Feature Matrix

Feature Module	System Operation	Target Performance Goal
UI Drag-and-Drop Canvas	Responsive graphical dashboard built with CustomTkinter.	Seamless file handling with no terminal or command-line friction.
Automated Pipeline Fallback	Conditional routing that catches 404 responses from threat databases.	Prevents dead ends when scanning new or uncataloged files.
Dynamic Trust Constraints	System-prompt filters that discard text from non-official web domains.	Prevents the AI from being tricked by malicious mirrors hosting forged hash lists.
P2P Offline Secure Mode	Manual input canvas that bypasses all web API layers for text-comparison matching.	Lets users securely check the integrity of private documents shared directly between two people.

Feature Module	System Operation	Target Performance Goal
Pipeline Visual Log	Interactive step-by-step progress tracker on the dashboard.	Shows internal operations live: Calculating Fingerprint → VirusTotal: 404 → Scraping Vendor Domain → Validating Integrity.

4. Engineering Technology Stack

Layer	Technology
Runtime language	Python 3.11+
GUI desktop toolkit	CustomTkinter (modern layout library)
Threat registry API client	VirusTotal Public API v3
AI orchestration framework	LangChain / LangGraph
Web search processing core	Tavily Search API (built for direct text-chunk extraction)
Temporary memory index	Local in-memory ChromaDB instance
AI core client	OpenAI API (`gpt-4o-mini`) at `temperature = 0.0` or `0.1` , to ensure strict textual extraction

5. Standard Project Target Structure

To keep the implementation modular and production-ready, the generated workspace must follow this structure:

```
veriblue_ai/
│
├── main.py                 # Application entry point & CustomTkinter UI loop
├── config.py               # API keys configuration (OpenAI, Tavily,
VirusTotal)
│
├── core/
│   ├── __init__.py
│   ├── hasher.py           # Stream-hashing logic (Step 0)
│   └── parser.py           # Binary header metadata extractor (Step 0)
│
└── engines/
    ├── __init__.py
    ├── virus_total.py      # Stage 1 API lookup & conditional logic handler
    └── web_rag.py          # Stage 2 LangChain & Tavily retrieval engine
```

6. Target Implementation Roadmap

The development engine must build the components sequentially across three phases:

1. **Phase 1: Local Engine Development.** Create the scripts that isolate internal app headers and calculate block-by-block file hash fingerprints locally.
2. **Phase 2: Network Execution Pipeline.** Set up the multi-stage logic loops that handle the conditional 404 transition from the threat aggregator to the live search engine.
3. **Phase 3: Asynchronous GUI Application.** Wrap the entire backend processing chain in an asynchronous dashboard built with CustomTkinter frames.

7. Execution Target Prompt for Antigravity

```
Act as an expert Python Software Architect and Senior Cybersecurity Engineer. Build a production-ready application layout for a desktop application based on the provided VeriBlue AI PRD specifications.

Program the codebase matching the modular target layout:
1. main.py: A clean CustomTkinter desktop UI with an active file drag-and-drop landing container box and an animated pipeline execution progress bar frame running asynchronously so large file processes never lock up UI elements.
2. core/hasher.py & parser.py: A deterministic file-streaming function using the native hashlib module to generate a 64-character SHA-256 hash string using 4096-byte chunk buffering alongside binary structural header metadata isolation.
3. engines/virus_total.py: A Stage 1 connection loop to the VirusTotal v3 API using the local calculated hash search key. If the API returns a malicious flag, route the JSON logs to an LLM context wrapper to build a plain-English explanation. If the API returns a 404 Not Found error status code, trap the exception smoothly and pass execution down to Stage 2.
4. engines/web_rag.py: A Stage 2 LangChain orchestration pipeline using Tavily to search for official software verification page readouts, appending domain-restricted site rules matching the file metadata. Text content must be fed into an ephemeral local vector cache to pass context blocks directly into a gpt-4o-mini client running at temperature=0.0.
5. A rigid system prompt enforcing that the AI must only return the clean 64-character verification hash string found within the official text or output 'NOT_FOUND'.
6. A strict traditional boolean match checkpoint comparing the initial locally calculated hash string to the dynamic AI fetched hash string, updating the UI window to display clear GREEN (Safe/Genuine) or RED (Tampered/Altered) verdict windows.
```